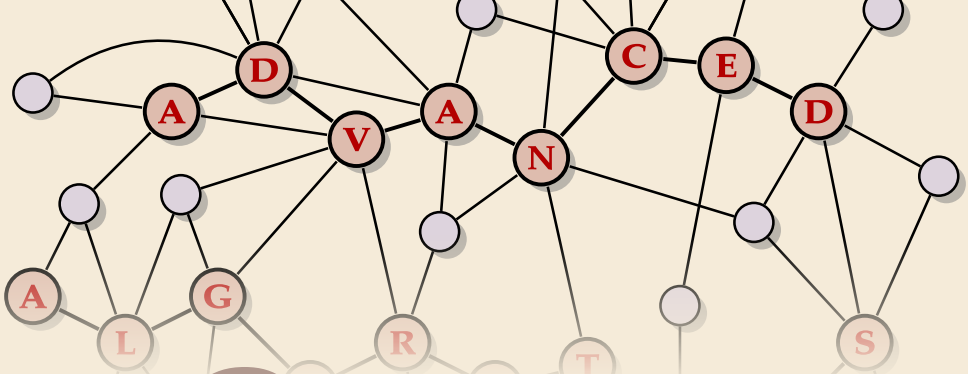


3 Pseudopolynomial Algorithms

Advanced Algorithms · Winter 2026–27

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
version `2026-10-24 03:38`

Outline

3 Pseudopolynomial Algorithms

3.1 Integer Problems

3.2 Knapsack

3.3 Strong NP-hardness

3.1 Integer Problems

Integer Problems

Definition 3.1 (Integer-Input Problem)

A problem $\Pi = (L, Y)$ for which (part of the) input is a *sequence of integers* is called an *integer-input problem*.

For any instance x of an integer-input problem, we write $MaxInt(x)$ for the largest integer occurring in the input encoding. ◀

(As before, integers are encoded in binary.)

Integer Problems

Definition 3.1 (Integer-Input Problem)

A problem $\Pi = (L, Y)$ for which (part of the) input is a *sequence of integers* is called an *integer-input problem*.

For any instance x of an integer-input problem, we write $MaxInt(x)$ for the largest integer occurring in the input encoding. ◀

(As before, integers are encoded in binary.)

Examples

- TravelingSalesman
- SubsetSum
- BinPacking
- ILP
- Knapsack

Clicker Question

How large can $MaxInt(x)$ be as a function of $n = |x|$?



A	$O(1)$	**D**	$O(n^c)$	**G**	$2^{O(n)}$
B	$O(\log n)$	**E**	$O(n^{\log n})$	**H**	$O(2^{2^n})$
C	$O(n)$	**F**	$O(2^n)$	**I**	unbounded



→ *sli.do/advalg*

Clicker Question

How large can $MaxInt(x)$ be as a function of $n = |x|$?



- **A** $O(1)$ ✓
- **B** $O(\log n)$ ✓
- **C** $O(n)$ ✓
- **D** $O(n^c)$ ✓
- **E** $O(n^{\log n})$ ✓
- **F** $O(2^n)$ ✓
- **G** ~~$2^{O(n)}$~~
- **H** ~~$O(2^{2^n})$~~
- **I** ~~unbounded~~



→ *sli.do/advalg*

Pseudopolynomial Algorithms

Definition 3.2 (Pseudopolynomial algorithm)

Let $\Pi = (L, Y)$ be an integer-input problem and A an algorithm that solves Π.
A has *pseudopolynomial time for* Π, if there is a polynomial p in two variables with

$$time_A(x) = O\Big(p\big(|x|, MaxInt(x)\big)\Big),$$

for every instance $x \in L$.

Pseudopolynomial Algorithms

Definition 3.2 (Pseudopolynomial algorithm)

$\neq$ quasi-polynomial ($= 2^{O(\log^c n)}$)

Let $\Pi = (L, Y)$ be an integer-input problem and A an algorithm that solves Π.
A has *pseudopolynomial time for* Π, if there is a polynomial p in two variables with

$$time_A(x) \quad = \quad O\Big(p\big(|x|, MaxInt(x)\big)\Big),$$

for every instance $x \in L$. ◀

Note: If $MaxInt(x) \leq h(|x|)$ for a polynomial h, then
$p\big(|x|, MaxInt(x)\big) \leq g(|x|)$ for a polynomial g.

Number-bounded restriction

Definition 3.3 (Number-bounded restriction)

Let Π be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing. The *h-restriction of* Π, written $\Pi_{MaxInt \leq h}$, is Π restricted to inputs x with $MaxInt(x) \leq h(|x|)$. ◀

Number-bounded restriction

Definition 3.3 (Number-bounded restriction)

Let Π be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing.
The *h-restriction of* Π, written $\Pi_{MaxInt \leq h}$, is Π restricted to inputs x with $MaxInt(x) \leq h(|x|)$.

Theorem 3.4 (Pseudopolynomial is polynomial for small h)

Let Π be an integer-input problem and A a pseudopolynomial algorithm for Π.
Then for every polynomial h there is a polytime algorithm for $\Pi_{MaxInt \leq h}$.

Proof:

A has running time $O(p(|x|, MaxInt(x)))$ $\quad n = |x|$

$x \in \mathcal{U}_{MaxInt \leq h} \Rightarrow MaxInt(x) \leq h(|x|) = O(n^c)$ for constant c

$\Rightarrow O(p(n, n^c)) = O(n^d)$ for constant d

Number-bounded restriction

Definition 3.3 (Number-bounded restriction)

Let Π be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing.
The *h-restriction of* Π, written $\Pi_{MaxInt \leq h}$, is Π restricted to inputs x with $MaxInt(x) \leq h(|x|)$.

Theorem 3.4 (Pseudopolynomial is polynomial for small h)

Let Π be an integer-input problem and A a pseudopolynomial algorithm for Π.
Then for every polynomial h there is a polytime algorithm for $\Pi_{MaxInt \leq h}$.

Proof:

Hence if Π is a decision problem then $\Pi_{MaxInt \leq h} \in \mathsf{P}$,
if Π is an optimization problem then $\Pi_{MaxInt \leq h} \in \mathsf{PO}$.

3.2 Knapsack

Knapsack (Optimization Version)



Definition 3.5 (Knapsack (Optimization Version))

Given: tuple $(w_1, \ldots, w_n; v_1, \ldots, v_n; b)$ of $2n + 1$ positive integers, $n \in \mathbb{N}$.
We call b the *capacity* of the knapsack, w_i the *weight* and v_i the *value* (profit) of the i-th object, $1 \leq i \leq n$.

Goal: The *optimization problem* KNAPSACK asks to find a subset $T \subseteq \{1, 2, \ldots, n\}$ of items ~~with maximal~~ $value(T) = \sum_{i \in T} v_i$ such that T fits into the knapsack, i. e., $\sum_{i \in T} w_i \leq b$.

◀

Recap: The 6 Steps of Dynamic Programming

⇝ see *Efficient Algorithms*

1. Define **subproblems** (and relate to original problem)
2. **Guess** (part of solution) ⇝ local brute force
3. Set up **DP recurrence** (for quality of solution)
4. Recursive implementation with **Memoization**
5. Bottom-up **table filling** (topological sort of subproblem dependency graph)
6. **Backtracing** to reconstruct optimal solution

- Steps 1–3 require insight / creativity / intuition;
  Steps 4–6 are mostly automatic / same each time

⇝ Correctness proof usually at level of DP recurrence


running time too! worst case time = #subproblems · time to find single best guess

Dynamic Programming Solution

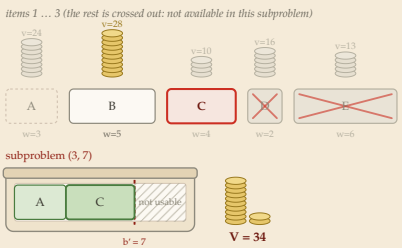
- **Subproblems:** (n', b') for $0 \le n' \le n$, $0 \le b' \le b$: only items $1 \le i \le n'$, capacity b'
- **Guess:** whether to include item n'
- **Recurrence:** $V[n', b']$ = max value in subproblem (n', b')

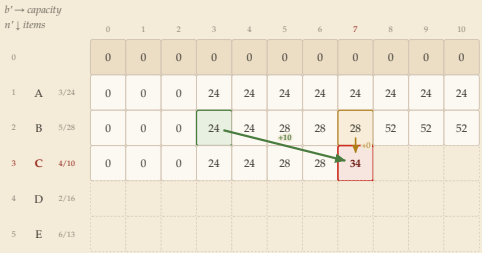
subproblems = $n \times b$ — bounded only by Marlin

bounded by input size

Dynamic Programming Solution

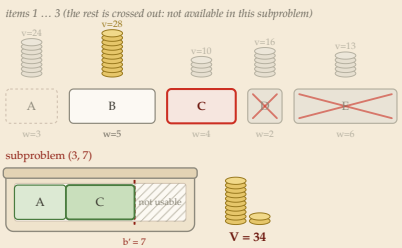
- **Subproblems:** (n', b') for $0 \le n' \le n, 0 \le b' \le b$: only items $1 \le i \le n'$, capacity b'
- **Guess:** whether to include item n'
- **Recurrence:** $V[n', b'] =$ max value in subproblem (n', b')

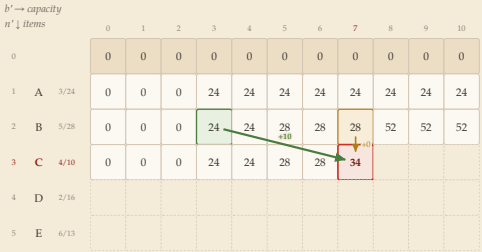




Dynamic Programming Solution

- **Subproblems:** (n', b') for $0 \le n' \le n, 0 \le b' \le b$: only items $1 \le i \le n'$, capacity b'
- **Guess:** whether to include item n'
- **Recurrence:** $V[n', b'] =$ max value in subproblem (n', b')

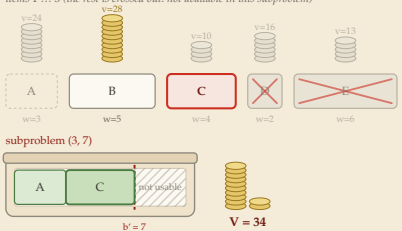


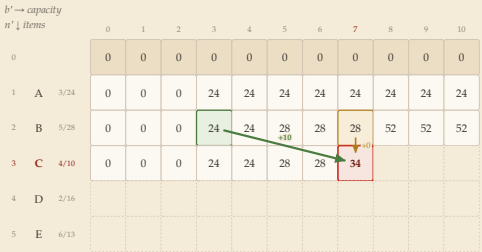


$$V[n', b'] = \begin{cases} 0 & \text{if } n' = 0, \\ V[n'-1, b'] & \text{if } n' \ge 1 \text{ and } w_{n'} > b', \\ \max\{V[n'-1, b'],\ v_{n'} + V[n'-1, b'-w_{n'}]\} & \text{otherwise.} \end{cases}$$

Dynamic Programming Solution

- **Subproblems:** (n', b') for $0 \le n' \le n, 0 \le b' \le b$: only items $1 \le i \le n'$, capacity b'
- **Guess:** whether to include item n'
- **Recurrence:** $V[n', b'] =$ max value in subproblem (n', b')





$$V[n', b'] = \begin{cases} 0 & \text{if } n' = 0, \\ V[n'-1, b'] & \text{if } n' \ge 1 \text{ and } w_{n'} > b', \\ \max\{V[n'-1, b'],\ v_{n'} + V[n'-1, b'-w_{n'}]\} & \text{otherwise.} \end{cases}$$

$\rightsquigarrow$ answer $V[n, b]$; $(n+1)(b+1)$ entries, $O(1)$ arithmetic operations each

Pseudopolynomial Knapsack

Theorem 3.6 (DP for Knapsack is pseudopolynomial)

For every instance I of KNAPSACK we have

$$time_{DPKP}(I) = O\big(|I| MaxInt(I) \log(1 + MaxInt(I))\big),$$

i. e., DPKP has pseudopolynomial time for KNAPSACK. ◀

Clicker Question

we used V[n',b']



A KNAPSACK instance has n items with *huge* weights and capacity (say $\approx 2^n$), but all *values* satisfy $v_i \leq n^2$.
Can we solve it in polynomial time?

- **A** Yes, with DP from the slides
- **B** Yes, with a different DP
- **C** No, KNAPSACK is NP-hard.
- **D** No idea.



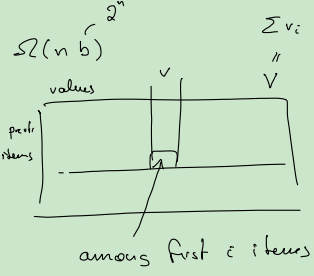
→ *sli.do/advalg*

Clicker Question



A KNAPSACK instance has n items with *huge* weights and capacity (say $\approx 2^n$), but all *values* satisfy $v_i \leq n^2$.
Can we solve it in polynomial time?

- **A** ~~Yes, with DP from the slides~~
- **B** Yes, with a different DP ✓
- **C** ~~No, KNAPSACK is NP-hard~~
- **D** No idea. ✓



to reach value EXACTLY v'
I need weight $W[n', v']$

→ *sli.do/advalg*

A Second DP for Knapsack: Over Values

- DPKP indexes the table by *capacity* $b' \leq b$ $\rightsquigarrow$ time $\propto b$
- Same idea works with *value* as the table index $\rightsquigarrow$ time $\propto V := \max_i v_i$

A Second DP for Knapsack: Over Values

- DPKP indexes the table by *capacity* $b' \le b$ $\rightsquigarrow$ time $\propto b$
- Same idea works with *value* as the table index $\rightsquigarrow$ time $\propto V := \max_i v_i$
- **Subproblems:** (n', v') for $0 \le n' \le n$, $0 \le v' \le nV$:
  only items $1 \le i \le n'$, total value *exactly* v'
- **Guess:** whether to include item n'
- **Recurrence:** $W[n', v']$ = minimum total weight of a subset of items $1..n'$
  with value exactly v' (∞ if none)

A Second DP for Knapsack: Over Values

- DPKP indexes the table by *capacity* $b' \le b$ $\rightsquigarrow$ time $\propto b$
- Same idea works with *value* as the table index $\rightsquigarrow$ time $\propto V := \max_i v_i$
- **Subproblems:** (n', v') for $0 \le n' \le n$, $0 \le v' \le nV$:
  only items $1 \le i \le n'$, total value *exactly* v'
- **Guess:** whether to include item n'
- **Recurrence:** $W[n', v']$ = minimum total weight of a subset of items $1..n'$
  with value exactly v' (∞ if none)

$$W[n', v'] = \begin{cases} 0 & \text{if } n' = 0 \text{ and } v' = 0, \\ \infty & \text{if } n' = 0 \text{ and } v' > 0, \\ W[n'-1, v'] & \text{if } n' \ge 1 \text{ and } v_{n'} > v', \\ \min\{ W[n'-1, v'],\ w_{n'} + W[n'-1, v' - v_{n'}] \} & \text{otherwise.} \end{cases}$$

A Second DP for Knapsack: Over Values

- DPKP indexes the table by *capacity* $b' \le b$ $\rightsquigarrow$ time $\propto b$
- Same idea works with *value* as the table index $\rightsquigarrow$ time $\propto V := \max_i v_i$
- **Subproblems:** (n', v') for $0 \le n' \le n$, $0 \le v' \le nV$:
  only items $1 \le i \le n'$, total value *exactly* v'
- **Guess:** whether to include item n'
- **Recurrence:** $W[n', v']$ = minimum total weight of a subset of items $1..n'$
  with value exactly v' (∞ if none)

$$W[n', v'] = \begin{cases} 0 & \text{if } n' = 0 \text{ and } v' = 0, \\ \infty & \text{if } n' = 0 \text{ and } v' > 0, \\ W[n'-1, v'] & \text{if } n' \ge 1 \text{ and } v_{n'} > v', \\ \min\{ W[n'-1, v'],\ w_{n'} + W[n'-1, v' - v_{n'}] \} & \text{otherwise.} \end{cases}$$

$\rightsquigarrow$ answer: $\max\{v' : W[n, v'] \le b\}$ (largest value that still fits)

Knapsack by values

Theorem 3.7 (DPKnapsack)

The value-indexed DP (*DPKnapsack*) solves KNAPSACK in time $O\left(n^2 \cdot V \cdot \log MaxInt(w)\right)$ with $V = \max_i v_i$. ◀

Proof:
$(n+1)(nV+1)$ entries, each a min of two sums of weights $\leq n \cdot MaxInt(w)$;
arithmetic on $O(\log(n\, MaxInt(w)))$-bit numbers. ■

Knapsack by values

Theorem 3.7 (DPKnapsack)

The value-indexed DP (*DPKnapsack*) solves Knapsack in time $O\left(n^2 \cdot V \cdot \log MaxInt(w)\right)$ with $V = \max_i v_i$. ◀

Proof:
$(n+1)(nV+1)$ entries, each a min of two sums of weights $\leq n \cdot MaxInt(w)$;
arithmetic on $O(\log(n\, MaxInt(w)))$-bit numbers. ■

$\rightsquigarrow$ polynomial in n and V, *independent of b and the weights*

Beyond Knapsack

- Similar trick works for some other NP-complete problems, e. g., Partition, MakingChange
- for yet other NP-complete problems, e. g., TravelingSalesman, no such algorithms seems to exist . . .

Beyond Knapsack

- Similar trick works for some other NP-complete problems, e. g., Partition, MakingChange
- for yet other NP-complete problems, e. g., TravelingSalesman, no such algorithms seems to exist . . .

  . . . can we give evidence that likely ***no*** *pseudopolynomial algorithm is* ***possible****?*

3.3 Strong NP-hardness

Strong Hardness

$\Pi = (L, Y)$ L = universe of sensible inputs $Y \subseteq L$ yes instances

Definition 3.8 (strongly NP-hard)

An integer-input problem Π is called *strongly* NP*-hard*, if there exists a polynomial p such that $\Pi_{MaxInt \leq h}$ is NP-hard. ◀

So: strongly NP-hard $\rightsquigarrow$ hard even for instances with "small" numbers.

Recall

Definition 3.3 (Number-bounded restriction)

Let Π be an integer-input problem and let $h : \mathbb{N} \to \mathbb{N}$ be weakly increasing. The *h-restriction of* Π, written $\Pi_{MaxInt \leq h}$, is Π restricted to inputs x with $MaxInt(x) \leq h(|x|)$.

Strongly NP-hard → no pseudopoly. algorithm

Theorem 3.9 (strongly NP-hard → no pseudopoly. algorithm)

Let $\mathsf{P} \neq \mathsf{NP}$ and $\Pi = (L, Y)$ a strongly NP-hard (integer-input) problem.
Then there exists no algorithm with pseudopolynomial time for Π.

Proof:

Π strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p such that $\Pi_{MaxInt \leq p}$ is NP-hard.

Strongly NP-hard → no pseudopoly. algorithm

Theorem 3.9 (strongly NP-hard → no pseudopoly. algorithm)

Let $\mathsf{P} \neq \mathsf{NP}$ and $\Pi = (L, Y)$ a strongly NP-hard (integer-input) problem.
Then there exists no algorithm with pseudopolynomial time for Π.

Proof:

Π strongly NP-hard $\quad\rightsquigarrow\quad$ $\exists$ polynomial p such that $\Pi_{MaxInt \leq p}$ is NP-hard.

Assume A is a pseudopolynomial algorithm for Π, i. e.,
$time_A(x) \leq q\big(|x|, MaxInt(x)\big)$ for a polynomial q.

Strongly NP-hard → no pseudopoly. algorithm

Theorem 3.9 (strongly NP-hard → no pseudopoly. algorithm)

Let $\mathsf{P} \neq \mathsf{NP}$ and $\Pi = (L, Y)$ a strongly NP-hard (integer-input) problem.
Then there exists no algorithm with pseudopolynomial time for Π.

Proof:

Π strongly NP-hard $\quad\rightsquigarrow\quad$ $\exists$ polynomial p such that $\Pi_{MaxInt \leq p}$ is NP-hard.

Assume A is a pseudopolynomial algorithm for Π, i. e.,
$time_A(x) \leq q\big(|x|, MaxInt(x)\big)$ for a polynomial q.

For $x \in \Pi_{MaxInt \leq p}$ we have $MaxInt(x) \leq p(|x|)$, so

$$time_A(x) \quad \leq \quad q\big(|x|, p(|x|)\big) \quad = \quad O\big(|x|^c\big) \qquad \text{for a constant } c.$$

Strongly NP-hard → no pseudopoly. algorithm

Theorem 3.9 (strongly NP-hard → no pseudopoly. algorithm)

Let $\mathsf{P} \neq \mathsf{NP}$ and $\Pi = (L, Y)$ a strongly NP-hard (integer-input) problem.
Then there exists no algorithm with pseudopolynomial time for Π.

Proof:

Π strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p such that $\Pi_{MaxInt \leq p}$ is NP-hard.

Assume A is a pseudopolynomial algorithm for Π, i. e.,
$time_A(x) \leq q\big(|x|, MaxInt(x)\big)$ for a polynomial q.

For $x \in \Pi_{MaxInt \leq p}$ we have $MaxInt(x) \leq p(|x|)$, so

$$time_A(x) \;\leq\; q\big(|x|, p(|x|)\big) \;=\; O\big(|x|^c\big) \qquad \text{for a constant } c.$$

$\rightsquigarrow$ A decides $\Pi_{MaxInt \leq p}$ in polynomial time $\rightsquigarrow$ $\Pi_{MaxInt \leq p} \in \mathsf{P}$.

Strongly NP-hard → no pseudopoly. algorithm

Theorem 3.9 (strongly NP-hard → no pseudopoly. algorithm)

Let $\mathsf{P} \neq \mathsf{NP}$ and $\Pi = (L, Y)$ a strongly NP-hard (integer-input) problem.
Then there exists no algorithm with pseudopolynomial time for Π.

Proof:

Π strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p such that $\Pi_{MaxInt \leq p}$ is NP-hard.

Assume A is a pseudopolynomial algorithm for Π, i. e.,
$time_A(x) \leq q\big(|x|, MaxInt(x)\big)$ for a polynomial q.

For $x \in \Pi_{MaxInt \leq p}$ we have $MaxInt(x) \leq p(|x|)$, so

$$time_A(x) \;\leq\; q\big(|x|, p(|x|)\big) \;=\; O\big(|x|^c\big) \qquad \text{for a constant } c.$$

$\rightsquigarrow$ A decides $\Pi_{MaxInt \leq p}$ in polynomial time $\rightsquigarrow$ $\Pi_{MaxInt \leq p} \in \mathsf{P}$.

Since $\Pi_{MaxInt \leq p}$ is NP-hard, this gives $\mathsf{P} = \mathsf{NP}$ ↯ $\rightsquigarrow$ no such A exists. ■

~~strongly NP-hard~~ ≠ strictly NP-hard

I misremembered the terms ... it's even worse!!

strongly NP-hard = our notion here, implies no pseudopolynomial algorithm (unless P=NP)

*strongly polynomial running time = #arithmetic operations bounded by the *number* of inputs (*independent* of encoding size)*

Example Strongly NP-Hard

Theorem 3.10

TravelingSalesman is strongly NP-hard.

Proof:

We show HamiltonianCycle $\leq_p$ TravelingSalesman$_{MaxInt \leq p}$ for $p(n) = n$.

Example Strongly NP-Hard

Theorem 3.10

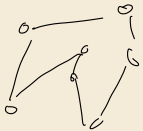
TravelingSalesman is strongly NP-hard.

Proof:

We show HamiltonianCycle $\leq_p$ TravelingSalesman$_{MaxInt \leq p}$ for $p(n) = n$.

Given a graph $G = (V, E)$ with $|V| = n$, output the TravelingSalesman instance (D, k) with

$$D_{u,v} = \begin{cases} 1 & \text{if } \{u, v\} \in E \\ 2 & \text{otherwise} \end{cases} \qquad \text{and} \qquad k = n.$$



Example Strongly NP-Hard

Theorem 3.10

TravelingSalesman is strongly NP-hard. ◀

Proof:

We show HamiltonianCycle $\leq_p$ TravelingSalesman$_{MaxInt \leq p}$ for $p(n) = n$.

Given a graph $G = (V, E)$ with $|V| = n$, output the TravelingSalesman instance (D, k) with

$$D_{u,v} = \begin{cases} 1 & \text{if } \{u, v\} \in E \\ 2 & \text{otherwise} \end{cases} \qquad \text{and} \qquad k = n.$$

$\rightsquigarrow$ round trip of length $\leq n$

$\iff$ all n edges of the trip have length 1

$\iff$ Hamiltonian cycle in G.

Example Strongly NP-Hard

Theorem 3.10

TravelingSalesman is strongly NP-hard. ◀

Proof:

We show HamiltonianCycle $\leq_p$ TravelingSalesman$_{MaxInt \leq p}$ for $p(n) = n$.

Given a graph $G = (V, E)$ with $|V| = n$, output the TravelingSalesman instance (D, k) with

$$D_{u,v} = \begin{cases} 1 & \text{if } \{u, v\} \in E \\ 2 & \text{otherwise} \end{cases} \qquad \text{and} \qquad k = n.$$

$\rightsquigarrow$ round trip of length $\leq n$

$\iff$ all n edges of the trip have length 1

$\iff$ Hamiltonian cycle in G.

$MaxInt(D, k) = \max(2, k) = n \leq |(D, k)|$, so every image in TravelingSalesman$_{MaxInt \leq p}$; the map is clearly polytime. ■

Clicker Question

Assume P ≠ NP. Which of these problems are *strongly* NP-hard?



A VertexCover (input: graph G, integer k)

B Knapsack

C SubsetSum

D TravelingSalesman

E BinPacking

F No idea.



→ *sli.do/advalg*

It's all about the encoding

Theorem 3.11 (strongly hard iff unary hard)

An integer-input problem is strongly NP-hard if, and only if, representing its instances with *unary encoding* for integers remains NP-hard. ◀

Proof:

π A strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p s.t. $A_{MaxInt \leq p}$ NP-hard

It's all about the encoding

Theorem 3.11 (strongly hard iff unary hard)

An integer-input problem is strongly NP-hard if, and only if, representing its instances with *unary encoding* for integers remains NP-hard. ◀

Proof:

A strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p s.t. $A_{MaxInt \leq p}$ NP-hard

For $x \in A_{MaxInt \leq p}$, unary encoding blows up $|x|$ only by polynomial factor.

$\rightsquigarrow$ A encoded with unary numbers NP-hard.

It's all about the encoding

Theorem 3.11 (strongly hard iff unary hard)

An integer-input problem is strongly NP-hard if, and only if, representing its instances with *unary encoding* for integers remains NP-hard. ◀

Proof:

A strongly NP-hard $\rightsquigarrow$ $\exists$ polynomial p s.t. $A_{MaxInt \leq p}$ NP-hard

For $x \in A_{MaxInt \leq p}$, unary encoding blows up $|x|$ only by polynomial factor.

$\rightsquigarrow$ A encoded with unary numbers NP-hard.

Conversely, let A with unary numbers be NP-hard.

With unary encoding, $MaxInt(x) \leq |x|$, so $A_{MaxInt \leq n} = A$ is NP-hard. ■

Clicker Question



Consider KNAPSACK, but with all numbers encoded in *unary*.
Which is true?

A It is in PO.

B It is NP-hard.

C Open problem.

D No idea.



→ *sli.do/advalg*

Clicker Question

Consider KNAPSACK, but with all numbers encoded in *unary*.
Which is true?



A It is in PO. ✓

B ~~It is NP-hard~~

C ~~Open problem.~~

D No idea. ✓



→ *sli.do/advalg*

Summary



Pseudopolynomial algorithms can be practically efficient if numbers are (really) small



Only applicable to few problems

- DP with $\geq$ *MaxInt* subproblems
- factoring polynomials (pseudopolynomial wrt. total degree)